

Synergistic Optimization for Efficient Large Language Model Serving: A Comprehensive Survey on Key-Value Cache Management, Speculative Decoding, and Quantization

1 Introduction

The deployment of Large Language Models (LLMs) in production environments is fundamentally constrained by the architectural inefficiencies inherent in autoregressive generation. Unlike the initial prompt processing phase, which is compute-bound and highly parallelizable, the decoding phase operates in a memory-bound regime where latency is dominated by the sequential transfer of model weights and Key-Value (KV) cache states from high-bandwidth memory to compute units [1]. This “memory wall” phenomenon creates a severe bottleneck, as the sequential nature of token production prevents the full utilization of modern accelerator parallelism, leading to suboptimal hardware efficiency and escalated operational costs. Consequently, the primary challenge in efficient serving shifts from maximizing floating-point operations per second (FLOPS) to optimizing memory bandwidth utilization and minimizing data movement overheads.

To address these constraints, this survey establishes a conceptual framework centered on the “Efficiency Trilemma,” which posits an inherent conflict between maximizing context length, maintaining generation accuracy, and minimizing inference latency. Traditional approaches often optimize these dimensions in isolation, yet such decoupled strategies frequently yield diminishing returns or introduce unacceptable accuracy degradation. For instance, while aggressive KV cache eviction can extend context windows, it risks losing critical semantic information required for coherent long-form generation [2]. Similarly, naive quantization schemes may reduce memory footprint but fail to account for activation outliers, leading to significant perplexity increases [3]. Therefore, achieving scalable deployment requires a synergistic optimization strategy that jointly navigates these trade-offs.

This survey focuses on a triad of complementary optimization pillars: KV cache management, speculative decoding, and quantization. First, advanced KV cache management techniques, such as PagedAttention, treat attention states as pageable memory blocks to eliminate internal fragmentation and enable non-contiguous allocation, thereby increasing batch concurrency by factors of 2–4× [4]. Furthermore, structural compression methods like sparse attention and layer-wise eviction policies [5] reduce the linear growth of memory consumption, allowing for longer contexts without proportional hardware expansion. Second, speculative decoding frameworks decouple proposal and verification steps, utilizing smaller drafter models or internal mechanisms to generate multiple tokens in parallel [6]. Recent advancements in tree-based verification [7] and retrieval-based drafting [8] have further enhanced acceptance rates, effectively breaking the sequential dependency of token generation. Third, quantization techniques compress model weights and activation states into low-bit representations, directly alleviating memory bandwidth pressure. Innovations in outlier mitigation and mixed-precision storage [9] ensure that compression ratios do not compromise generation fidelity.

Crucially, this survey argues that the future of efficient LLM serving lies not in the independent application of these techniques, but in their co-design. The interplay between quantization noise and speculative acceptance rates, or the coordination between compressed cache hierarchies and speculative verification pipelines, represents a frontier of significant untapped potential. By synthesizing memory-centric, algorithm-centric, and representation-centric approaches, we aim to provide a comprehensive taxonomy that guides the development of unified serving systems capable of balancing

cost, accuracy, and performance in real-world applications.

2 Advanced Key-Value Cache Management and Memory Optimization

2.1 Dynamic Memory Allocation and Virtual Memory Abstractions

The exponential growth of Key-Value (KV) cache memory consumption with sequence length and batch size presents a fundamental bottleneck in large language model (LLM) serving, often leading to severe internal fragmentation and underutilization of GPU memory. Traditional serving systems rely on contiguous pre-allocation strategies that reserve maximum potential memory for each request, resulting in significant waste when actual sequence lengths vary dynamically. To address this inefficiency, recent innovations have adapted virtual memory abstractions from operating systems to decouple the logical address space of the KV cache from its physical memory allocation. This paradigm shift enables non-contiguous storage of cache blocks, effectively eliminating internal fragmentation and allowing for fine-grained memory management that scales with actual demand rather than worst-case estimates.

A pioneering approach in this domain is PagedAttention, which treats the KV cache as pageable memory blocks analogous to OS pages [4]. By managing physical memory at the block level, PagedAttention allows the system to allocate non-contiguous physical frames for a logically contiguous sequence, achieving near-zero memory waste. This flexibility significantly increases the effective batch size and throughput, particularly for workloads with highly variable sequence lengths. However, implementing PagedAttention requires substantial modifications to attention kernels to handle scattered memory accesses, potentially introducing overhead and complicating the integration of optimized linear algebra libraries.

In contrast, vAttention proposes an alternative architecture that retains contiguous virtual memory layouts while leveraging OS support for on-demand physical page allocation [10]. This approach decouples virtual and physical memory without requiring custom kernel rewrites, allowing serving systems to utilize state-of-the-art attention implementations like FlashAttention out-of-the-box. Empirical evaluations suggest that vAttention can improve decode throughput by up to $1.99\times$ compared to PagedAttention-based systems by reducing programming overhead and maintaining optimal memory access patterns [10]. The trade-off lies in the reliance on OS paging mechanisms, which may introduce latency variability during page faults if not carefully managed.

Beyond device memory management, efficient serving of long contexts often necessitates swapping inactive KV cache blocks between high-bandwidth GPU VRAM and slower host storage or NVMe. Techniques such as microbatch swapping orchestrate this data movement during pipeline bubbles to hide latency, effectively expanding the usable context window beyond physical device limits [11]. Furthermore, intelligent prefetching strategies, inspired by learning-augmented paging algorithms, can predict future access patterns to proactively load relevant cache blocks, mitigating the performance penalty of memory disaggregation [12; 13]. These systems often employ adaptive policies that balance the cost of data transfer against the probability of cache hits, ensuring that memory bandwidth is utilized efficiently even under fluctuating capacity constraints [14].

Looking forward, the convergence of hardware-aware paging and predictive memory management promises to further relax memory constraints. Future research directions include the development of hierarchical memory pools that seamlessly integrate GPU, CPU, and distributed storage tiers, governed by schedulers that optimize for both latency Service Level Objectives (SLOs) and total cost of ownership [15]. As models scale to handle million-token contexts, the synergy between virtual

memory abstractions and speculative execution will become critical, requiring unified frameworks that manage page faults and draft verification concurrently to maintain high goodput.

2.2 Token Pruning and Saliency-Aware Eviction Strategies

While the preceding section detailed system-level innovations that optimize memory allocation through virtual abstractions and paging, token pruning and saliency-aware eviction strategies address the linear memory growth of the Key-Value (KV) cache by algorithmically discarding tokens deemed less critical for future generation. Unlike the structural compression methods discussed in the subsequent section, which modify model architecture to intrinsically reduce dimensionality, these techniques operate dynamically at inference time. They leverage the observation that attention distributions are highly skewed, with a small subset of “heavy hitter” tokens dominating the attention scores [16]. The foundational hypothesis driving this domain is the “persistence of importance,” which posits that tokens exhibiting high influence at one step are likely to remain pivotal in subsequent steps [17]. Consequently, early approaches like [16] and [17] employ eviction policies that retain a fixed budget of recent tokens and those with the highest accumulated attention scores, effectively treating KV cache management as a dynamic submodular optimization problem.

However, relying solely on attention scores presents limitations, as it overlooks the magnitude of value vectors which also contributes significantly to the attention output. [18] demonstrates that incorporating the ℓ_1 norm of value vectors alongside attention scores yields a more robust importance metric, leading to the Value-Aware Token Pruning (VATP) strategy which outperforms score-only baselines on long-context benchmarks. Furthermore, the definition of “criticality” is increasingly recognized as query-dependent rather than static. [19] introduces a query-aware selection mechanism that estimates the criticality of KV cache pages based on the current query vector, loading only the top- k relevant pages for attention computation. Similarly, [20] extends this logic to the channel dimension, pruning redundant channels based on query-driven significance to minimize attention weight loss. Retrieval-based formulations further generalize this concept; [21] frames token selection as a Maximum Inner-Product Search (MIPS) problem using product quantization, fetching only the most relevant KV pairs for the current decoding step, thereby decoupling memory footprint from context length without permanent data loss.

A particularly innovative direction involves exploiting the hierarchical nature of information flow within Transformers. Research reveals a “pyramidal information funneling” phenomenon where lower layers require broad context coverage while upper layers progressively consolidate attention onto fewer critical tokens [22; 5]. Methods like [22] and [5] leverage this by implementing layer-wise adaptive budget allocation, aggressively pruning tokens in higher layers while preserving density in lower ones, achieving significant memory reduction with negligible accuracy degradation. Despite these advances, aggressive eviction risks irreversible information loss, potentially causing hallucinations or context collapse. To mitigate this, hybrid approaches such as [23] and [24] propose retaining evicted tokens in low-precision formats or maintaining full caches only for specific “retrieval heads,” thus balancing strict memory constraints with the need for contextual fidelity. While these strategies effectively reduce the token count, they operate orthogonally to techniques that compress the representation of retained tokens; the next section explores how structural compression and low-rank approximations reduce the dimensionality and redundancy of attention states without discarding historical information. Future research must address the stability of these policies under distribution shifts and explore learned eviction strategies that can adapt to diverse task semantics without manual hyperparameter tuning.

2.3 Structural Compression and Low-Rank Approximations

Structural compression and low-rank approximations represent a paradigm shift in Key-Value (KV) cache management, moving beyond token-level eviction to intrinsically reduce the dimensionality and redundancy of attention states. Unlike dynamic pruning strategies that discard historical information, these techniques exploit the inherent algebraic structures within Transformer architectures—specifically inter-layer correlations and channel redundancies—to compress representations without altering the fundamental autoregressive process. A primary avenue for structural compression involves sharing key-value heads across adjacent layers or query groups. While Grouped-Query Attention (GQA) reduces the number of unique heads per layer, recent advancements propose Cross-Layer Attention (CLA), which extends this sharing mechanism vertically across the network depth. [25] demonstrates that sharing KV heads between adjacent layers can halve the cache footprint with negligible accuracy degradation, offering a Pareto improvement over traditional Multi-Query Attention by leveraging the observation that semantic representations often stabilize across deep layers. Similarly, [26] optimizes the transformation from Multi-Head to Grouped-Query Attention by explicitly accounting for low-rank characteristics and Rotary Positional Embeddings (RoPE), ensuring that compression errors are minimized during the architectural modification.

Complementing head-sharing, channel shrinking and tensor decomposition address redundancy within the feature dimension rather than the sequence length. Methods such as [27] operate on the premise that KV states in middle-to-deep layers exhibit high similarity. By disentangling state vectors into magnitude and direction components, [27] interpolates directions while preserving lengths, effectively merging adjacent layers to create compact composite representations. This “merge-and-split” strategy allows for significant memory reduction while maintaining the fidelity required for long-context reasoning. Furthermore, low-rank matrix decomposition techniques have been adapted to approximate the quantization error in compressed caches. [9] introduces a hybrid approach where a low-rank matrix approximates the residual error of ultra-low precision quantization, while a sparse matrix corrects outliers, thereby achieving near-lossless compression at high ratios. This synergistic integration of quantization, low-rank approximation, and sparsity highlights the potential of mathematical transformations to overcome the limitations of uniform compression.

Despite their efficacy, these structural approaches face challenges regarding hardware compatibility and dynamic adaptability. Architectural modifications like CLA often require retraining or specific model configurations, limiting their applicability as plug-and-play solutions for pre-trained models. Conversely, post-hoc decomposition methods like those in [9] and [27] offer greater flexibility but may introduce computational overhead during the reconstruction phase. Emerging trends suggest a convergence of structural compression with query-aware selection mechanisms. For instance, combining the layer-wise budget allocation of [22] with low-rank approximations could enable adaptive compression that respects the “pyramidal information funneling” phenomenon, where lower layers require broader context retention than upper layers. Future research must address the trade-off between the static nature of structural changes and the dynamic requirements of varying input distributions, potentially through hybrid systems that dynamically switch between full-rank and low-rank cache modes based on runtime complexity estimates.

2.4 Mixed-Precision Quantization and Outlier Mitigation

Building on the structural compression and low-rank approximations discussed previously, quantization offers a complementary orthogonal axis for optimization by reducing the numerical precision

of KV cache elements rather than their dimensionality or sequence length. While uniform low-bit quantization provides a straightforward path to memory reduction, it frequently falters in the presence of activation outliers—extreme values in specific channels or tokens that disproportionately distort quantization ranges and degrade attention fidelity. Consequently, recent research has pivoted toward mixed-precision schemes and outlier mitigation strategies that adaptively allocate bit-widths based on salience, thereby preserving numerical stability while maximizing compression ratios.

A predominant strategy involves identifying and isolating salient tokens or channels for high-precision storage while aggressively compressing the remainder. [23] demonstrates that evicting low-salience KV pairs entirely can induce hallucinations and context loss; instead, retaining them in low precision while keeping critical tokens in high precision recovers performance. Similarly, [28] utilizes normalized attention scores to identify salient tokens, employing a channel-separable tokenwise quantization scheme that decouples saliency metrics from full attention computation to maintain compatibility with FlashAttention. This importance-aware approach is further refined by [18], which argues that attention scores alone are insufficient proxies for token importance. By incorporating the ℓ_1 norm of value vectors, this work proposes Value-Aware Token Pruning (VATP), ensuring that tokens with high semantic magnitude are preserved even if their immediate attention scores are modest.

Parallel to token-level adaptation, transformation-based methods aim to smooth the activation distribution to facilitate uniform low-bit quantization without requiring per-token exceptions. [29] introduces a rotation-based framework that applies orthogonal transformations to the hidden states, effectively spreading outlier magnitudes across all channels. This renders the KV cache amenable to end-to-end 4-bit quantization, including weights, activations, and cache states, with negligible perplexity degradation. Complementing this, [30] optimizes these rotation matrices via Cayley optimization on validation data, demonstrating that learned rotations outperform random ones in minimizing quantization error for hard-to-quantize models. Furthermore, [31] proposes “CushionCache,” a novel technique where specific KV prefixes are tuned to regularize subsequent token activations, preventing the emergence of outliers at the source rather than merely mitigating them post-hoc.

Beyond mixed-precision and smoothing, advanced encoding schemes exploit inter-channel dependencies to achieve extreme compression, bridging the gap between quantization and the structural techniques mentioned earlier. [32] challenges the assumption of channel independence, proposing Coupled Quantization (CQ) that jointly encodes multiple channels to leverage their shared entropy, successfully maintaining model quality at merely 1-bit per channel. Similarly, [21] adapts product quantization from database retrieval, clustering KV vectors into codebooks to enable efficient Maximum Inner-Product Search (MIPS) during decoding, thus reducing memory footprint without sacrificing retrieval accuracy. Despite these advances, challenges remain in balancing the computational overhead of dynamic bit allocation or rotation operations against the latency gains from reduced memory bandwidth. These quantization-driven memory savings provide the essential foundation for the systemic paradigm shift toward disaggregated architectures explored in the next section, where optimized cache footprints enable efficient distribution across heterogeneous memory tiers.

2.5 Disaggregated Architectures and Distributed Cache Systems

Disaggregated architectures represent a paradigm shift in LLM serving, decoupling compute and memory resources to transcend the capacity constraints of single-node GPU clusters. By separating the compute-intensive prefill phase from the memory-bound decoding phase, these systems enable independent scaling of resources, thereby eliminating the interference that plagues collocated execution models. [33] demonstrates that assigning prefill and decoding to distinct GPU pools allows for co-optimized parallelism strategies tailored to the specific latency requirements of Time-to-First-Token (TTFT) and Time-Per-Output-Token (TPOT), achieving up to $4.48\times$ higher request throughput under stringent Service Level Objectives (SLOs). Similarly, [34] leverages disaggregation alongside chunked prompt partitioning to mitigate scheduling hotspots, reducing resource consumption by 38% while significantly lowering job completion times.

Beyond phase separation, recent innovations treat the Key-Value (KV) cache as a first-class, shareable resource distributed across heterogeneous memory tiers. [15] introduces a KVCache-centric disaggregated architecture that aggregates underutilized CPU DRAM and SSDs within a GPU cluster to form a massive, elastic memory pool. This approach facilitates the handling of ultra-long context windows by dynamically orchestrating data movement between high-bandwidth device memory and cost-effective host storage, achieving a 525% throughput increase in long-context scenarios. Complementing this, [35] proposes DistAttention, a distributed algorithm that segments KV caches into manageable units for processing across data center-scale GPU groups, supporting context lengths up to 1.9M tokens while maintaining end-to-end throughput improvements of $1.03\text{--}2.4\times$.

The efficacy of distributed systems is further enhanced by intelligent data localization and reuse mechanisms. In Retrieval-Augmented Generation (RAG) and multi-turn dialogue, redundant computation can be eliminated by treating KV caches as persistent knowledge artifacts. [36] organizes intermediate states of retrieved knowledge into a hierarchical tree structure, caching them across GPU and host memory to reduce TTFT by $4\times$. Extending this concept to multi-turn interactions, [37] employs a hierarchical caching system with layer-wise pre-loading and asynchronous saving to enable efficient attention reuse across conversation turns, reducing inference costs by up to 70%. Furthermore, [38] and [39] optimize prefix sharing by detecting overlapping text segments across requests, utilizing prefix-aware trees and two-phase partitioning to accelerate self-attention kernels by up to $4.8\times$.

However, disaggregation introduces non-trivial challenges regarding communication latency and consistency. While Remote Direct Memory Access (RDMA) offers low-latency connectivity, naive implementations often suffer from protocol overheads. [40] addresses this by providing a remote dynamic memory cache with customizable SLOs, effectively bridging the performance gap between local DRAM and remote memory pools. Yet, the trade-off between computation and communication remains critical; as noted in [33], placement strategies must carefully balance network bandwidth against compute utilization to prevent the disaggregation overhead from negating concurrency gains. Future directions point towards “serverless” KV cache markets where cache blocks are migrated dynamically based on real-time access patterns, potentially leveraging predictive prefetching techniques akin to [41] to hide remote memory latencies. Ultimately, the evolution of disaggregated systems hinges on the tight co-design of scheduling algorithms, network protocols, and hierarchical storage managers to create a unified, scalable fabric for next-generation LLM inference.

3 Speculative Decoding and Parallel Verification Frameworks

3.1 Drafting Strategies and Model Architectures

The fundamental efficacy of speculative decoding hinges on the drafting mechanism’s ability to generate high-quality token sequences with minimal latency overhead. This subsection categorizes drafting strategies into three primary archetypes: independent auxiliary models, internal architectural modifications, and retrieval-based mechanisms, each presenting distinct trade-offs between model alignment, computational cost, and system complexity.

The most prevalent approach employs independent drafter models, typically smaller variants distilled from the target model to ensure distributional alignment. [42] demonstrates that standard distillation is often insufficient; instead, utilizing on-policy data generation and tailoring divergence functions to the decoding strategy significantly enhances acceptance rates, yielding speedups of 10–45% over baseline speculative decoding. Similarly, [43] explores distilling large instruction-tuned models into diverse, smaller architectures, confirming that compact models can approximate target behaviors effectively when trained on high-diversity instruction sets. However, maintaining separate model instances incurs substantial memory overhead and complicates deployment, particularly in resource-constrained environments.

To mitigate the memory footprint of independent drafters, recent innovations embed drafting capabilities directly within the target model architecture. [44] introduces auxiliary decoding heads attached to the frozen backbone, enabling the parallel prediction of multiple subsequent tokens via tree-based attention. This single-model approach eliminates the need for a distinct drafter, achieving over $2.2\times$ speedup with negligible quality loss. Advancing this concept, [45] proposes sequentially dependent “Hydra heads” that leverage preceding draft tokens to improve speculation accuracy, outperforming independent head architectures. Furthermore, [46] utilizes early exiting mechanisms, where a shallow sub-network serves as the drafter and the remaining layers act as the verifier, bridging the representation gap with lightweight adapters. These internal mechanisms optimize memory locality but require specific fine-tuning recipes, potentially limiting their applicability to off-the-shelf models.

Distinct from parametric approaches, retrieval-based and n-gram drafting strategies leverage non-parametric knowledge stores to propose tokens. [8] harnesses existing knowledge reservoirs to retrieve relevant token sequences based on context, offering a plug-and-play acceleration method that requires no additional training. This is particularly effective for repetitive or domain-specific text where exact matches are frequent. Complementing this, [47] exploits identical text spans between decoding results and available references (e.g., retrieved documents) to copy and verify tokens in parallel, achieving over $2\times$ speedup in scenarios with high context-output overlap. While these methods avoid training costs, their performance is inherently bounded by the coverage and latency of the underlying retrieval system.

Emerging trends point toward hybrid architectures that combine the strengths of these paradigms. For instance, [48] constructs a phrase candidate pool from the verification process to guide small model drafting, effectively creating a feedback loop that refines draft quality dynamically. Additionally, [49] integrates recurrent dependencies into lightweight draft heads, merging the sequential modeling strength of RNNs with the parallel verification of transformer-based speculators. Future research must address the challenge of adaptive drafting, where the system dynamically selects or synthesizes drafting strategies based on real-time workload characteristics and hardware constraints to maximize goodput across diverse application domains.

3.2 Parallel Verification Algorithms and Acceptance Criteria

Building upon the diverse drafting strategies categorized previously, this subsection examines the parallel verification mechanisms that transform speculative proposals into guaranteed acceleration. While the preceding section detailed how independent, internal, and retrieval-based drafters generate candidate tokens, the efficacy of these proposals is ultimately determined by the verification stage’s ability to reconcile the draft distribution with the target model’s true distribution. The core objective here is to accelerate token generation while strictly preserving the target model’s output distribution, ensuring lossless acceleration through rigorous probabilistic acceptance mechanisms. The foundational approach relies on a modified rejection sampling scheme, as formalized in [50], which accepts a draft token x_t with probability $\min(1, q(x_t)/p(x_t))$, where q and p denote the draft and target probabilities, respectively. This mathematical guarantee ensures that the marginal distribution of the generated sequence remains identical to that of the target model, regardless of the draft model’s quality, provided the support of q is contained within p .

While standard rejection sampling verifies tokens sequentially, recent advancements have shifted towards tree-based parallel verification structures to maximize the hardware utilization enabled by the drafting mechanisms discussed earlier. [51] introduces a paradigm where draft tokens are organized into a tree, allowing the target model to evaluate multiple branching paths simultaneously via specialized attention masks. This approach transforms the verification from a linear process into a parallel one, significantly increasing the expected number of accepted tokens per forward pass. The efficacy of such tree-based methods is further enhanced by dynamic programming algorithms that optimize the tree structure itself, as demonstrated in [7], which tailors the tree topology to specific hardware constraints and decoding temperatures to maximize speedup.

Beyond structural optimization, the acceptance criteria themselves have evolved to handle complex decoding scenarios and improve upon simple linear verification. Multi-candidate verification strategies, explored in [52], extend the verification scope beyond a single linear trajectory by sampling and verifying multiple candidate sequences in batches. This method improves acceptance rates by evaluating diverse potential futures, effectively mitigating the risk of early rejections that halt speculative chains. Furthermore, [53] proposes verifying tokens based on joint likelihood scores rather than independent token probabilities. By approximating the joint distribution of the large model, this approach allows for the acceptance of the longest prefix sub-sequence with valid perplexity, offering a more flexible trade-off between generation quality and inference speed.

To further refine the balance between computational cost and acceptance gains, adaptive mechanisms have been integrated directly into the verification pipeline to dynamically adjust speculation depth. [54] formulates the choice of candidate length as a Markov Decision Process, employing a threshold policy where speculation halts when the predicted rejection probability exceeds a learned bound. This prevents wasted computation on low-confidence drafts. Similarly, [46] introduces confidence-based early exiting within the verification pipeline, halting the drafting process once uncertainty rises, thus optimizing the cost-benefit ratio of verification.

Despite these advances, challenges remain in balancing verification overhead with acceptance gains, particularly under varying workload characteristics. Emerging trends suggest a move towards “verify-then-correct” paradigms, such as [55], which integrates semi-autoregressive inference to enable simultaneous generation and correction. However, the static configurations often used in these verification schemes can struggle to navigate the inherent trade-offs in dynamic production environments. This limitation necessitates a deeper exploration of adaptive control mechanisms that optimize end-to-end goodput by aligning speculation parameters with real-time hardware con-

straints, a topic addressed in the following subsection. Future research must also address the robustness of these acceptance criteria under quantization noise and distribution shifts, ensuring that the theoretical guarantees of distributional equivalence hold in practical, resource-constrained serving environments.

3.3 Adaptive Control and Dynamic Resource Orchestration

While speculative decoding theoretically offers substantial acceleration, its practical efficacy is contingent upon the dynamic alignment of speculation parameters with real-time workload characteristics and hardware constraints. Static configurations often fail to navigate the inherent trade-off between the computational cost of verification and the probability of token acceptance, necessitating adaptive control mechanisms that optimize end-to-end goodput rather than raw throughput. A primary challenge lies in determining the optimal speculation depth; fixed-length drafting can lead to diminishing returns when acceptance rates fluctuate due to input complexity or distribution shifts. To address this, recent works formulate candidate length selection as a sequential decision problem. For instance, SpecDec++ models this as a Markov Decision Process, deriving a threshold policy where speculation halts when the predicted rejection probability exceeds a learned bound, thereby adaptively modulating draft lengths to maximize efficiency [54]. Similarly, SmartSpec introduces a goodput-centric framework that dynamically adjusts speculation strides from zero to maximum based on observed system load and acceptance accuracy, preventing latency inflation during periods of low speculation utility [56].

Beyond linear depth, the structural configuration of speculative trees requires hardware-aware optimization to fully exploit parallel verification capabilities. The geometry of the token tree—specifically its width and depth—must be tailored to the underlying accelerator’s memory bandwidth and compute parallelism. Sequoia addresses this by employing a dynamic programming algorithm to identify optimal tree structures that maximize decoding speed for specific hardware platforms, coupled with a hardware-aware optimizer that auto-tunes tree dimensions to mitigate bottlenecks [7]. This approach recognizes that a tree structure optimal for a high-bandwidth GPU may be suboptimal for memory-constrained edge devices or offloading scenarios. Furthermore, the efficacy of these structures is heavily dependent on the alignment between the draft and target distributions, which can drift in production environments. Online Speculative Decoding (OSD) mitigates this by leveraging surplus cluster compute to continuously retrain draft models on incoming query distributions, ensuring that adaptive policies operate on accurate predictive baselines even under non-stationary workloads [57].

The orchestration of resources extends to the scheduling domain, where speculation interacts with batch management and request prioritization. In multi-tenant settings, indiscriminate speculation can exacerbate tail latency if long-speculation requests block shorter ones. Efficient Interactive LLM Serving with Proxy Model-based Sequence Length Prediction proposes a Speculative Shortest-Job-First (SSJF) scheduler that utilizes lightweight proxy models to predict output lengths, enabling priority scheduling that reduces average job completion times significantly compared to First-Come-First-Served policies [58]. This highlights the synergy between predictive control and queue management. Moreover, in Retrieval-Augmented Generation (RAG) contexts, adaptive control must coordinate speculation with retrieval latency. RaLMSpec demonstrates this by integrating an optimal speculation stride scheduler with asynchronous verification and prefetching, automatically exploiting acceleration potentials while hiding retrieval overheads [59].

Looking forward, the frontier of adaptive control lies in unified, learning-augmented orchestrators

that jointly optimize quantization bit-widths, tree topology, and batching strategies. Current approaches often treat these dimensions orthogonally, yet their interdependencies are profound; for example, aggressive quantization may alter acceptance distributions, requiring compensatory adjustments in speculation depth. Future systems will likely employ reinforcement learning agents, akin to those used in hardware prefetching [60] or caching [61], to navigate this high-dimensional configuration space autonomously. Such systems must balance the overhead of online adaptation against performance gains, ensuring robustness against adversarial inputs and extreme distribution shifts while adhering to strict Service Level Objectives (SLOs).

3.4 Synergistic Integration with Quantization and Memory Systems

Building on the adaptive control mechanisms and unified orchestrators envisioned in the previous section, the synergistic integration of speculative decoding with quantization and memory management emerges as a critical frontier for overcoming the efficiency trilemma in large language model serving. While speculative decoding accelerates generation by decoupling proposal and verification [6], its practical efficacy is intrinsically coupled to the numerical fidelity of the underlying models and the accessibility of attention states. A primary challenge lies in the impact of low-bit quantization on draft accuracy; aggressive compression of weights and activations in drafter models can induce distributional shifts that degrade token acceptance rates. Recent advancements suggest that uniform low-bit quantization often fails to preserve the subtle probability margins required for high acceptance, necessitating mixed-precision strategies. For instance, frameworks like [62] demonstrate that co-designing quantization schemes (e.g., W4A8KV4) with system kernels can mitigate dequantization overheads while maintaining the alignment necessary for speculative verification. Similarly, [63] highlights that fine-grained mixed-precision quantization is essential to sustain throughput gains without compromising the statistical equivalence between draft and target distributions.

Beyond weight quantization, the interaction between speculative pipelines and compressed Key-Value (KV) cache introduces complex consistency constraints that directly impact the hardware-aware optimizations discussed earlier. Speculative verification typically requires random access to historical KV states to evaluate multiple candidate tokens in parallel; however, compressed caches employing eviction or ultra-low-bit quantization may obscure the precise attention scores needed for rigorous rejection sampling. Methods such as [64] address this by integrating dynamic sparse KV cache retrieval into the drafting hierarchy, using the original model weights as an intermediate draft to bridge the gap between compressed memory states and accurate verification. Conversely, retaining evicted tokens in low precision, as proposed in [23], offers a robust mechanism to recover context lost during eviction, thereby stabilizing the acceptance probability of speculative branches that depend on long-range dependencies. The tension between memory bandwidth and compute intensity is further exacerbated in speculative settings; while quantization reduces the memory footprint of the KV cache [65], the irregular memory access patterns inherent in tree-based verification [7] can negate these benefits if data movement is not carefully orchestrated.

Emerging trends point toward joint optimization frameworks that dynamically modulate both speculation depth and quantization granularity based on real-time hardware constraints and workload characteristics, realizing the holistic vision of learning-augmented orchestrators. The concept of “coupled quantization,” which exploits inter-channel dependencies to achieve extreme compression rates (e.g., 1-bit per channel) [32], presents a promising avenue for reducing the memory bandwidth bottleneck that often limits speculative batch sizes. However, the integration of such extreme compression with speculative decoding requires novel verification algorithms capable of tolerating

higher noise floors in the attention computation. Future research must therefore focus on developing theoretical bounds for acceptance rates under joint quantization and speculation errors, as well as designing hardware-aware schedulers that can adaptively switch between high-precision verification and low-precision drafting to maximize goodput. Ultimately, the convergence of these techniques promises to unlock scalable serving architectures where memory capacity no longer dictates the feasible context length or concurrency, provided that the intricate trade-offs between numerical precision, memory locality, and algorithmic correctness are rigorously managed.

4 Quantization Techniques for Model Compression and Accelerated Inference

4.1 Outlier Mitigation and Activation-Aware Quantization Strategies

The simultaneous quantization of weights and activations in Large Language Models (LLMs) to ultra-low bit-widths is fundamentally impeded by the presence of activation outliers—channels exhibiting magnitudes orders of magnitude larger than the mean. These outliers dominate the dynamic range, forcing quantization scales that render the majority of values indistinguishable, thereby causing severe accuracy degradation in standard Post-Training Quantization (PTQ). To address this, recent research has pivoted from naive uniform quantization toward sophisticated outlier mitigation strategies that mathematically smooth activation distributions or structurally isolate salient features.

A dominant paradigm involves smoothness-inducing transformations that migrate quantization difficulty from activations to weights via offline, mathematically equivalent channel-wise scaling. [66] pioneeringly demonstrated that by absorbing activation outliers into adjacent weight matrices, one can enable hardware-friendly INT8 inference for both weights and activations without retraining. This approach leverages the observation that weights are inherently easier to quantize than activations. Building on this, [67] introduced Learnable Equivalent Transformation (LET), which optimizes these scaling factors within a differentiable framework to minimize block-wise reconstruction error, while [68] proposed reordering channels into clusters to mitigate inter-channel variance before quantization. Further advancing rotational invariance, [29] and [30] utilize Hadamard or learned orthogonal rotations to render weight and activation matrices incoherent, effectively spreading outlier magnitudes across all channels to facilitate end-to-end 4-bit quantization. Similarly, [69] employs adaptive channel reassembly to disassemble outlier channels into sub-channels, rebalancing magnitude distributions prior to merging.

Complementary to smoothing, sparse-high-precision retention strategies adopt a “Dense-and-Sparse” decomposition, isolating salient outliers for high-precision storage while aggressively quantizing the remaining dense parameters. [70] and [71] identify that a tiny fraction of weights (often $<1\%$) contributes disproportionately to quantization error; by storing these outliers in FP16 and compressing the rest to 3-4 bits, these methods achieve near-lossless compression. [72] extends this logic to weight-only quantization by searching for per-channel scaling factors that protect salient weights identified through activation analysis, significantly preserving generalization without backpropagation. In the realm of extreme compression, [73] and [69] explore partial binarization, where salient weights are filtered and retained in higher precision to prevent the collapse of linguistic reasoning capabilities inherent in naive 1-bit schemes.

Architectural regularization offers a proactive alternative, modifying the model during training to suppress outlier emergence. Techniques such as clipped softmax and gated attention have been shown to constrain activation ranges, facilitating native integer-only inference. Furthermore, [74] highlights the critical role of “pivot tokens” in the initial sequence positions, advocating for their

lossless retention to bound quantization error propagation. While smoothing techniques like [66] offer plug-and-play compatibility, they may struggle with layer-specific activation spikes in GLU-based architectures, a limitation addressed by [75] through module-specific quantization exemption. Conversely, sparse retention methods like [70] introduce irregular memory access patterns that can complicate kernel optimization, necessitating specialized decoding algorithms. Future research must therefore focus on co-designing quantization-aware training objectives that intrinsically regularize activation distributions, reducing the reliance on post-hoc corrections and enabling robust, uniform low-bit inference across heterogeneous hardware substrates.

4.2 Specialized Quantization Schemes for Key-Value Cache Compression

Building directly on the outlier mitigation and mixed-precision philosophies established for static weights, the Key-Value (KV) cache presents a distinct yet parallel challenge as the primary memory bottleneck in autoregressive decoding. Unlike static parameters, KV states exhibit dynamic distributions characterized by heavy outliers and inter-channel dependencies that evolve with sequence length, necessitating specialized quantization schemes that diverge from standard weight-only compression. While early efforts applied uniform low-bit integer quantization, naive implementation often precipitates severe accuracy degradation due to the heightened sensitivity of attention scores to quantization noise. Consequently, recent methodologies have shifted toward asymmetric and fine-grained strategies tailored to the unique statistics of attention states. For instance, [65] demonstrates that key and value caches possess distinct statistical properties, advocating for per-channel quantization for keys and per-token quantization for values to achieve tuning-free 2-bit compression with negligible perplexity loss. Similarly, [76] theoretically decouples the quantization strategies for keys and values, employing non-uniform grids and dedicated outlier handling to attain up to 10x compression ratios.

Mirroring the “Dense-and-Sparse” decomposition discussed previously, a critical innovation in KV cache optimization is the integration of mixed-precision mechanisms to safeguard salient information. [23] argues that indiscriminate eviction or uniform low-bit quantization risks hallucinations and context loss; instead, it proposes retaining critical KV pairs in high precision while compressing evicted tokens to low bit-widths, thereby balancing memory efficiency with generation reliability. This aligns with [28], which utilizes normalized attention scores to identify salient tokens dynamically, applying aggressive compression only to non-critical states while maintaining a strong baseline through channel-separable tokenwise quantization. Furthermore, extending the concept of coupled representations beyond independent scalar limits, [32] leverages the joint entropy of multiple channels, demonstrating that KV caches can be compressed to effectively 1-bit per channel by encoding coupled activations, challenging the conventional wisdom that such extreme compression is infeasible for attention states.

Beyond scalar quantization, structural compression via low-rank approximation and sparse decomposition offers complementary gains, echoing the architectural regularization strategies used for weights. [9] introduces a hybrid recipe combining ultra-low precision quantization with low-rank matrix approximation and sparse error correction, achieving near-lossless 4-bit compression that mitigates the autoregressive accumulation of errors. This approach contrasts with [27], which operates in the depth dimension by merging similar KV states across adjacent layers based on magnitude and direction disentanglement, effectively reducing redundancy without altering the numerical precision of individual entries. Additionally, the rotation-based techniques introduced for weight quantization are now being adapted for KV caches; [29] and [30] illustrate that rotating the hidden states can suppress outliers, enabling end-to-end 4-bit inference including the KV cache

without requiring high-precision retention layers.

Despite these algorithmic advances, significant challenges remain in harmonizing compression ratios with hardware efficiency, a gap that the subsequent section addresses in depth. While methods like [65] and [62] provide optimized kernels for 4-bit and 2-bit KV caches, the overhead of dequantization and the complexity of mixed-precision memory access patterns can erode latency benefits, particularly in high-throughput serving scenarios. Future research must therefore co-design quantization algorithms with memory hierarchies, potentially leveraging emerging hardware support for non-uniform data types. Moreover, the interaction between KV quantization and speculative decoding warrants deeper investigation, as quantization noise in the cache may disproportionately affect the acceptance rates of draft tokens. Ultimately, the trajectory points toward adaptive, query-aware compression frameworks that dynamically modulate precision based on runtime semantic criticality rather than static heuristics, setting the stage for the hardware-aware execution pipelines detailed next.

4.3 Hardware-Aware Kernel Design and Integer-Only Inference Pipelines

The transition from theoretical compression ratios to tangible inference acceleration hinges critically on hardware-aware kernel design and the construction of integer-only execution pipelines. While quantization reduces model footprint, realized speedups are often negated by the overhead of dequantization and the persistence of floating-point bottlenecks in standard linear algebra libraries. To bridge this gap, recent advancements focus on fusing dequantization directly into General Matrix Multiply (GEMM) operations, thereby minimizing global memory bandwidth usage and hiding latency through register-level parallelism. For instance, [62] introduces a compute-aware weight re-ordering strategy and leverages register-level parallelism to drastically reduce the runtime overhead of W4A8 quantization, demonstrating that efficient kernel design is paramount for cloud-scale serving. Similarly, [77] employs asynchronous memory access and bespoke task scheduling to support batched workloads with 4-bit weights, achieving near-optimal memory-bound performance that scales effectively with batch size.

Beyond optimizing mixed-precision workflows, a paradigm shift toward fully integer-only execution engines is emerging to eliminate floating-point units entirely, particularly for edge deployment. Frameworks such as [78] replace resource-intensive dequantization processes with efficient Lookup Table (LUT) based operations, enabling sub-4-bit weight quantization to yield actual computational reduction rather than mere memory savings. This approach is complemented by [79], which constructs a highly optimized backend to remove quantization and dequantization overheads, facilitating end-to-end INT8 pipelines that significantly outperform FP16 baselines on diverse hardware. Furthermore, [63] capitalizes on native low-bit integer operators available in modern GPUs, utilizing mixed-precision and fine-grained quantization to maximize serving throughput while maintaining accuracy. These systems collectively illustrate that achieving theoretical speedups requires co-designing quantization schemes with the underlying instruction set architectures, such as Intel’s VNNI or NVIDIA’s Tensor Cores, as highlighted in early industrial attempts to leverage INT8/VNNI instructions for Transformer models [80].

Memory alignment and data movement strategies further dictate the efficacy of low-bit inference. Irregular bit-widths, such as 2-bit or 3-bit representations, pose significant challenges for memory coalescing and address translation. [71] addresses this by storing outliers in a sparse format alongside dense low-bit weights, requiring specialized kernels to handle the heterogeneous data layout efficiently. In contrast, [29] utilizes rotation matrices to distribute outliers uniformly, enabling

end-to-end 4-bit integer arithmetic without the need for high-precision retention, thus simplifying the kernel design requirements. Additionally, [81] introduces heuristic dataflow adaptations and double buffering to optimize flat GEMM operations common in LLM decoding, mitigating performance losses caused by static dataflows and under-utilized computation. The integration of these techniques suggests a future trajectory where compilers automatically generate hardware-specific kernels that adapt to dynamic bit-widths and sparsity patterns. As shown in [82], compiler-based approaches like Apache TVM with quantization-aware dialects are essential for porting these optimizations across heterogeneous platforms, from CPUs to GPUs. Ultimately, the synergy between algorithmic innovation and system-level kernel optimization defines the frontier of efficient LLM serving, where the elimination of floating-point operations and the mastery of memory hierarchies will dictate the scalability of next-generation language models.

4.4 Advanced Training Paradigms and Mixed-Precision Co-Design

While the hardware-aware kernels and integer-only pipelines discussed previously successfully translate theoretical compression ratios into tangible speedups, they primarily optimize static, post-training quantized models. However, Post-Training Quantization (PTQ) often encounters an accuracy wall at ultra-low bit-widths (e.g., ≤ 4 -bit) due to the inability to redistribute quantization error across the network. To overcome this limitation without sacrificing the efficiency gains established by system-level co-design, advanced training paradigms integrating Quantization-Aware Training (QAT) with Parameter-Efficient Fine-Tuning (PEFT) have emerged as critical strategies for recovering performance in extreme compression regimes. A pivotal development in this domain is the synergy between low-bit quantization and Low-Rank Adaptation (LoRA), exemplified by [83], which demonstrates that backpropagating gradients through frozen 4-bit weights into high-precision adapters can match full-precision fine-tuning performance while drastically reducing memory footprint. This approach leverages novel data types like NormalFloat (NF4) and double quantization to optimize the information-theoretic bounds of weight representation. Building on this, [84] addresses the imbalance between the degrees of freedom in quantization and adaptation by introducing group-wise operators, allowing the model to integrate auxiliary weights naturally into the quantized structure without post-hoc merging errors. Similarly, [85] enhances generality by employing LoRA-wise learned step sizes, enabling simultaneous quantization and fine-tuning that yields linearly quantized weights superior to static PTQ configurations.

Beyond adapter-based methods, holistic QAT frameworks are pushing the boundaries of sub-4-bit precision to complement the low-bit integer operators utilized in modern serving engines. [86] proposes a two-phase strategy involving block-wise all-parameter training followed by end-to-end quantization parameter optimization, achieving near-lossless 2-bit quantization for 70B parameter models with minimal accuracy degradation. This contrasts with [87], which synergizes QAT with self-distillation via a Confidence-Aware Kullback-Leibler Divergence objective, effectively utilizing the teacher model’s own distribution to guide the student quantized model toward higher fidelity in sub-4-bit regimes. Furthermore, [88] explores the frontier of 1-bit quantization by introducing a specialized QAT framework with matrix decomposition-based initialization, proving that robust training processes can preserve over 83% of non-quantized performance even with binary weights. These methods collectively indicate that retraining specific components or the entire network with quantization noise simulation is essential for breaking the PTQ accuracy barrier and fully leveraging the computational benefits of the integer-only pipelines described earlier.

Parallel to these training refinements, mixed-precision co-design has become indispensable for managing activation outliers and KV cache bottlenecks, bridging the gap between algorithmic precision

and hardware constraints. Rather than uniform compression, [89] advocates for salience-driven mixed-precision allocation, where bit-widths are assigned based on weight clustering and element-wise salience, ensuring that critical information is preserved in higher precision while maintaining hardware-friendly grouped memory layouts. This concept extends to the KV cache, where [23] proposes an importance-aware mixed-precision scheme that retains salient tokens in high precision while compressing evicted or less important pairs to low-bit formats, thereby mitigating the hallucination risks associated with aggressive cache pruning. The integration of these quantization schemes with serving systems is further optimized in [62], which employs a W4A8KV4 co-design to minimize dequantization overheads through compute-aware weight reordering and register-level parallelism, effectively unifying the training-aware precision strategies with the kernel-level optimizations. Looking forward, the convergence of adaptive bit-allocation, learned rotation strategies like those in [30], and dynamic LoRA integration suggests a shift toward “any-precision” serving architectures [90] that can dynamically modulate model granularity based on real-time latency and accuracy constraints, setting the stage for the next generation of adaptive serving systems.

5 System-Level Integration and Synergistic Co-Design

5.1 Collaborative Precision and Speculation Strategies

The synergistic integration of quantization and speculative decoding represents a frontier in optimizing Large Language Model (LLM) serving, where the tension between memory efficiency and generation fidelity must be carefully managed. While quantization reduces the memory footprint and bandwidth requirements for both target and draft models, it inherently introduces numerical perturbations that can degrade the alignment between their output distributions, thereby reducing token acceptance rates. Recent empirical studies indicate that aggressive low-bit precision, particularly below 4-bit, can significantly impact the probability distribution matching essential for lossless acceleration [91]. Consequently, naive application of quantization to speculative pipelines often yields sub-optimal speedups, as the computational savings from low-bit arithmetic are offset by increased verification failures and the need for more frequent target model invocations.

To mitigate this, co-design strategies focusing on quantization-aware drafting have emerged. Methods like [87] demonstrate that aligning the draft model with the target model via knowledge distillation, specifically using on-policy data generation, can robustly maintain high acceptance rates even when both models operate under compressed regimes. Furthermore, the concept of mixed-precision speculative pipelines offers a promising architectural compromise. By employing ultra-low-bit quantization (e.g., INT4 or lower) for the draft model to maximize drafting speed, while retaining higher precision (e.g., FP16 or INT8) for the target verifier, systems can balance throughput and accuracy. This approach is supported by findings in [63], which shows that fine-grained mixed-precision quantization can boost serving throughput by up to 7.7x with negligible accuracy loss, suggesting that selective precision retention in the verification stage is critical for preserving the mathematical equivalence of the generation process.

Dynamic adaptation mechanisms further refine this interplay by modulating quantization bit-widths and speculation depths in real-time based on observed acceptance dynamics. [57] proposes continually updating draft models on observed query distributions to mitigate distribution shifts, a strategy that is equally applicable to adaptive quantization where bit-widths are lowered during high-confidence generation phases and increased when uncertainty rises. Additionally, the integration of outlier mitigation techniques is paramount; since activation outliers disproportionately affect quantization error and consequently draft accuracy, methods such as rotation-based smooth-

ing [29] or salience-driven mixed precision [89] must be incorporated into the drafting pipeline to ensure stable token proposals. Theoretical frameworks treating quantization as perturbation [92] suggest that controlling the variance of these perturbations is essential for maintaining the rejection sampling thresholds required for valid speculative execution.

Looking forward, the convergence of extreme compression and speculative execution necessitates novel verification protocols that are resilient to quantization noise. Future research should explore joint optimization objectives that simultaneously minimize quantization error and maximize speculative acceptance, potentially leveraging reinforcement learning to navigate the multi-dimensional trade-off space of bit-width, draft length, and hardware constraints. As hardware support for sub-4-bit operations matures, the development of end-to-end differentiable speculative frameworks that account for quantization-aware gradients will be crucial for unlocking the full potential of synergistic serving systems.

5.2 Hierarchical Memory Orchestration for Compressed State Management

Bridging the gap between the algorithmic synergies of quantization and speculative decoding discussed previously and the system-level SLO guarantees required in subsequent sections, the integration of quantized Key-Value (KV) cache compression with speculative decoding pipelines necessitates a paradigm shift from static memory allocation to dynamic, hierarchical memory orchestration. As context lengths extend into the hundreds of thousands of tokens, the memory footprint of attention states often exceeds high-bandwidth device memory, forcing systems to leverage heterogeneous storage tiers ranging from GPU VRAM to CPU DRAM and even NVMe SSDs. Effective orchestration in this regime requires synchronizing the irregular access patterns of speculative verification with the latency-hiding mechanisms of compressed data movement. Recent advancements demonstrate that unified memory managers must simultaneously handle the temporal sparsity of draft token buffers and the spatial redundancy of quantized KV states. For instance, systems like [64] leverage hierarchical speculative decoding where a dynamic sparse KV cache serves as an intermediate draft model, effectively reducing the data volume required for verification steps. This approach highlights the critical interplay between compression ratios and acceptance rates; aggressive quantization schemes such as those in [65] or [9], which employ low-rank approximations and sparse error correction, must be carefully calibrated to ensure that the noise introduced does not disproportionately degrade the drafter’s alignment with the target model, thereby undermining the speculative speedup.

A central challenge in this domain is the coordination of data prefetching with the probabilistic nature of speculative trees, a problem exacerbated by the decoding noise constraints identified earlier. Traditional demand paging fails to hide the latency of decompressing and transferring KV blocks from host memory during the narrow verification windows of speculative decoding. To address this, latency-hiding prefetching algorithms tailored for compressed streams have emerged. [93] proposes a mechanism where essential KV cache entries are speculated via minimal rehearsal, allowing the system to prefetch only critical tokens from host memory rather than the entire context, thus mitigating fetch overhead in offloading scenarios. Similarly, [41] illustrates how majority-based prefetching in disaggregated memory settings can drastically reduce tail latencies, a principle that translates directly to predicting speculative trajectories in LLM serving. By anticipating which branches of the speculative tree are most likely to be accepted, the system can overlap the dequantization of low-bit KV caches (e.g., 4-bit or 2-bit as in [62] and [28]) with the computation of the verification step, effectively masking the bandwidth bottleneck.

Furthermore, the architecture of elastic memory pools plays a pivotal role in supporting disaggregated serving models where prefill and decoding are separated, setting the stage for the dynamic resource allocation strategies detailed in the following section. [33] and [15] advocate for decoupling these phases to eliminate interference, utilizing distributed KV cache pools that span CPU DRAM and SSDs. In such architectures, the orchestration layer must manage the migration of quantized cache blocks between nodes without disrupting the speculative workflow. The synergy between compression and speculation is further exemplified by [17], which maintains a fixed memory budget by persisting only pivotal tokens, a strategy that complements speculative execution by ensuring that the most salient context—critical for high acceptance rates—resides in the fastest memory tier. However, trade-offs remain; while methods like [22] exploit layer-wise information funneling to reduce cache size dynamically, integrating such variable-sized cache structures with the fixed-width parallelism of speculative verification kernels requires sophisticated runtime scheduling. These complexities underscore the necessity for the advanced, co-tuned orchestration frameworks discussed next, which employ reinforcement learning-based controllers, akin to [60], to autonomously tune bit-widths, speculation depths, and prefetching aggressiveness in real-time, navigating the multi-dimensional trade-off space to maximize goodput under strict Service Level Objectives.

5.3 SLO-Driven Dynamic Resource Scheduling and Co-Tuning

Achieving strict Service Level Objectives (SLOs) in large language model (LLM) serving necessitates a paradigm shift from static resource allocation to dynamic, co-tuned orchestration that simultaneously navigates the trade-offs between quantization precision, speculative decoding depth, and cache retention policies. The inherent variability in LLM inference latency, driven by stochastic output lengths and input complexity, renders fixed configurations suboptimal for maintaining tail latency bounds while maximizing throughput. Recent advancements propose runtime frameworks that treat serving parameters as continuous control variables within a multi-objective optimization landscape. For instance, [94] introduces an uncertainty-aware ecosystem that quantifies input-induced latency variations, correlating text uncertainty with output length to dynamically prioritize requests and offload computation, thereby mitigating SLO violations under high-traffic conditions. Similarly, [95] proactively selects model variants and resource allocations to balance accuracy and cost against latency constraints, demonstrating significant reductions in both SLO violations and operational expenses compared to standard autoscalers.

The core challenge lies in the coupled nature of these optimization dimensions: aggressive quantization may reduce memory bandwidth pressure but degrade the acceptance rate of speculative tokens, while overly conservative cache eviction policies can precipitate out-of-memory errors during long-context generation. To address this, learning-based controllers have emerged as a promising solution. [56] formulates the selection of speculation length as a goodput-driven decision process, dynamically adjusting the number of drafted tokens based on real-time system load and observed acceptance probabilities to prevent the latency inflation often seen in static speculative decoding. Extending this logic, [54] models candidate length selection as a Markov Decision Process, employing a threshold policy derived from acceptance predictions to adaptively terminate speculation, yielding substantial speedups across diverse datasets. Furthermore, reinforcement learning approaches offer robustness against non-stationary workloads; [96] utilizes queueing models to account for cold start latencies in capacity planning, while [97] employs model-driven queueing methods to auto-scale resources for latency-sensitive serverless functions at the edge, ensuring fair-share allocation during overload scenarios.

Predictive modeling also plays a critical role in proactive resource management. [98] leverages

distributed tracing to model execution times across serverless application workflows, optimizing memory configurations to meet specific SLOs while minimizing costs. In the context of caching, [13] demonstrates that even minimal one-bit predictions regarding page reoccurrence can significantly enhance paging algorithms, suggesting that lightweight predictive signals can effectively guide KV cache eviction and swapping strategies without incurring heavy computational overhead. The integration of these techniques creates a synergistic effect: by jointly tuning quantization bit-widths and speculation depths, systems can operate closer to the theoretical efficiency frontier. For example, reducing precision for low-salience tokens while maintaining high precision for critical attention heads allows for larger batch sizes, which in turn influences the optimal speculation depth. However, current approaches often optimize these dimensions in isolation or rely on simplified assumptions about workload stationarity. Future research must focus on unified control planes that employ holistic reinforcement learning agents, akin to [60] in hardware prefetching, to learn joint policies that adapt to complex, multi-modal workload shifts while providing formal guarantees on SLO compliance.

5.4 Disaggregated Architectures and Heterogeneous Compute Collaboration

Building on the unified control planes and autonomous agents proposed for navigating multi-dimensional trade-offs, disaggregated architectures represent the physical realization of these dynamic strategies, shifting from monolithic deployments to decoupled compute-intensive prefill, memory-bound decoding, and speculative drafting across heterogeneous hardware tiers. This separation facilitates the specialized resource allocation envisioned by holistic controllers, mapping distinct phases of the inference pipeline to hardware best suited for their specific computational characteristics. A primary manifestation of this trend is the disaggregation of prefill and decoding tasks. Systems like [33] assign these phases to separate GPU clusters, eliminating the interference caused by colocation and enabling independent optimization of Time-to-First-Token (TTFT) and Time-Per-Output-Token (TPOT). By decoupling these phases, [33] achieves significant goodput improvements, serving up to 4.48x more requests under strict latency constraints compared to state-of-the-art baselines. Similarly, [15] implements a KVCache-centric disaggregated architecture that leverages underutilized CPU, DRAM, and SSD resources to create a massive, elastic cache pool, effectively addressing the memory capacity bottlenecks inherent in long-context serving.

The synergy between disaggregation and heterogeneous compute collaboration is further amplified when integrated with speculative decoding, allowing the dynamic speculation depths discussed previously to be executed on cost-effective accelerators. In this framework, the drafting phase, which is often less compute-intensive but latency-sensitive, can be offloaded to CPUs, NPUs, or edge devices, while the verification phase remains on high-bandwidth GPUs. [99] demonstrates the viability of collaborative edge computing, partitioning models across heterogeneous devices to reduce latency by up to 50% through adaptive device selection. Furthermore, [72] advocates for phase-aware partitioning on heterogeneous clusters, utilizing adaptive quantization to match model segments with devices of varying capabilities, thereby maximizing throughput without compromising quality. This approach aligns with the principles of [100], which optimizes workflow deployment across tiered infrastructures to minimize serving costs while meeting accuracy targets.

A critical enabler of these architectures, particularly when managing the elastic cache pools and distributed states required by dynamic SLO-aware policies, is the efficient management of distributed KV caches. As context windows expand, the cache often exceeds single-node memory, necessitating network-centric delivery mechanisms. [35] introduces DistAttention, a distributed algorithm that segments KV caches for processing across data center GPUs, supporting context lengths up to 1.9

million tokens. To mitigate the latency of remote memory access, [101] employs an elastic memory pool with a global scheduler that enhances cache reuse via a prompt tree-based locality policy, significantly reducing job completion times. Moreover, [40] and [41] illustrate how RDMA-enabled remote memory can serve as a high-performance cache tier, with [41] specifically optimizing prefetching to reduce tail latencies by over 22x in disaggregated settings. The integration of fault tolerance into these streams is also vital; [11] utilizes KV-cache streaming to enable microbatch swapping and state replication, ensuring robustness against node failures without sacrificing performance.

Despite these advancements, challenges remain in balancing communication overheads against computational gains, a complexity that reinforces the need for the predictive modeling and reinforcement learning agents highlighted in prior sections. The trade-off between the latency introduced by networked cache access and the throughput benefits of specialized hardware requires precise scheduling. Future directions point toward fully automated co-design frameworks, such as [102], which dynamically reconfigure cluster resources based on energy-performance trade-offs. Additionally, the emergence of serverless paradigms like [103] suggests a future where locality-enhanced checkpoint loading and live migration further abstract hardware heterogeneity, allowing systems to seamlessly scale drafting and verification tasks across diverse cloud and edge resources. Ultimately, the convergence of disaggregated storage, heterogeneous compute, and intelligent scheduling defines the next frontier of scalable, efficient LLM serving.

6 Benchmarking Methodologies, Evaluation Metrics, and Empirical Trends

6.1 Standardized Performance Metrics and Latency Characterization

Evaluating the efficacy of efficient Large Language Model (LLM) serving architectures demands a shift from aggregate throughput metrics to a granular, phase-specific latency taxonomy that reflects end-user Quality of Experience (QoE). The autoregressive generation process inherently decouples into two distinct computational regimes: the compute-bound prefill phase and the memory-bound decoding phase. Consequently, standardized evaluation must rigorously distinguish between Time-to-First-Token (TTFT), which governs user-perceived responsiveness, and Inter-Token Latency (ITL), which dictates the smoothness of text streaming. Recent advancements in disaggregated architectures, such as [33] and [15], explicitly optimize these phases independently, necessitating metrics that capture the interference patterns when these phases are collocated versus separated. For instance, [104] demonstrates that without chunked prefilling, long prompts can induce head-of-line blocking that drastically inflates TTFT for concurrent decoding requests, a phenomenon invisible to average throughput measurements.

Beyond mean latency, characterizing the tail of the latency distribution is critical for Service Level Objective (SLO) compliance in production environments. The stochastic nature of token generation lengths and hardware variability creates heavy-tailed latency distributions that average metrics fail to capture. As highlighted in [94], input uncertainty directly correlates with output length variance, leading to unpredictable latency spikes that violate strict SLOs. To mitigate this, scheduling policies like Speculative Shortest-Job-First (SSJF) have been proposed to reduce tail latency by prioritizing shorter jobs, thereby preventing queuing delays for latency-sensitive requests [58]. Furthermore, the concept of “goodput”—defined as the rate of successfully completed requests within their SLO bounds—has emerged as a superior metric to raw throughput. [56] argues that speculative decoding, while increasing raw token generation rates, can paradoxically degrade goodput under high load if the verification overhead exceeds the acceptance gains, emphasizing the need for dynamic speculation lengths tuned to real-time system load.

The evaluation framework must also account for the memory-bound characteristics of decoding, where performance is dictated by bandwidth rather than compute intensity. The “AI and Memory Wall” [1] posits that as model sizes scale, memory bandwidth becomes the primary bottleneck, rendering compute-centric metrics insufficient. Techniques like PagedAttention [4] alleviate memory fragmentation, yet their impact is best measured through the lens of memory utilization efficiency alongside latency percentiles (e.g., P95, P99). Moreover, speculative decoding strategies introduce a complex trade-off between computation and latency; [105] provides a theoretical framework quantifying this via the match rate ($p_{correct}$), suggesting that optimal benchmarking must jointly analyze the cost of rejected drafts against the latency reduction of accepted tokens. Finally, holistic QoE metrics, as formalized in [106], integrate TTFT, ITL, and stability into a unified score, arguing that optimizing server-side aggregate metrics often comes at the expense of individual user experience during bursty traffic. Future benchmarking methodologies must therefore adopt multi-dimensional profiles that simultaneously track phase-specific latencies, tail distributions, and goodput under varying degrees of memory pressure and workload uncertainty to truly gauge the synergistic benefits of KV cache management, speculation, and quantization.

6.2 Multi-Dimensional Accuracy and Safety Evaluation Frameworks

While the preceding analysis established that system-level metrics like goodput and tail latency are meaningless if model utility degrades, quantifying this utility requires moving beyond perplexity, which has long served as the canonical but increasingly insufficient proxy for evaluating efficient serving systems. Relying exclusively on perplexity often masks critical degradations in functional capabilities, particularly when aggressive techniques such as quantization, pruning, or KV cache eviction are applied. A robust evaluation framework must therefore transcend surface-level linguistic fluency to encompass downstream task robustness, parametric knowledge retention, and safety alignment. Recent empirical studies indicate that while perplexity may remain stable under moderate compression, performance on complex reasoning and long-context retrieval tasks can deteriorate significantly [107]. For instance, methods like [108] demonstrate that prompt compression can paradoxically improve task performance by enhancing information density, a nuance entirely invisible to perplexity metrics. Consequently, benchmark suites must prioritize task-specific accuracy across diverse domains, including coding, mathematical reasoning, and multi-hop question answering, to truly gauge the preservation of model utility [109].

Furthermore, the integrity of parametric knowledge within compressed models requires specialized scrutiny, as algorithmic efficiencies directly impact the fidelity of stored information. Aggressive quantization schemes, particularly those targeting ultra-low bit-widths (e.g., 2-3 bits), risk erasing fine-grained factual associations stored in model weights. The [70] and [71] frameworks highlight that while sparse-high-precision retention can mitigate perplexity shifts, they do not guarantee the preservation of specific world knowledge. To address this, evaluation protocols should incorporate diagnostics like LAMA (Language Model Analysis) to detect targeted forgetting patterns induced by weight or KV cache compression [107]. Similarly, the [110] paper introduces the Adversarial Compression Ratio (ACR) as a novel metric to assess whether compressed models retain the capacity to memorize and reproduce training data under adversarial prompting, offering a critical lens for copyright and privacy compliance in efficient serving.

Perhaps most critically, the intersection of efficiency and safety remains under-explored, yet it is vital for ensuring that acceleration does not compromise ethical constraints. Compression artifacts can disproportionately affect a model’s alignment, potentially amplifying biases or increasing susceptibility to jailbreaking attacks. The study [111] provides seminal evidence that while moderate

quantization (e.g., 4-bit) often preserves trustworthiness dimensions, extreme compression or unstructured pruning can severely degrade safety guards, leading to increased toxicity and bias even when benign task performance appears intact. This divergence necessitates a dedicated safety evaluation pillar that measures toxicity scores, fairness metrics across protected groups, and robustness against adversarial prefixes specifically for quantized and speculative systems. Additionally, in the context of speculative decoding, it is imperative to verify distributional fidelity. As noted in [50], speculative methods must mathematically guarantee equivalence to the target distribution; however, approximate verification or quantization-induced noise in the draft model can introduce subtle shifts in generation style or probability masses that standard benchmarks miss. Therefore, future evaluation frameworks must integrate statistical tests for distributional divergence alongside traditional accuracy metrics to ensure that acceleration does not come at the cost of generative consistency or ethical alignment. These rigorous quality and safety assessments provide the necessary foundation for the subsequent discussion on how such algorithmic choices interact with diverse hardware substrates to determine real-world viability.

6.3 Hardware-Aware Profiling and Cross-Platform Benchmarking

Effective optimization of Large Language Model (LLM) serving necessitates a rigorous understanding of how algorithmic efficiencies translate across disparate hardware substrates. The efficacy of techniques such as KV cache quantization and speculative decoding is intrinsically coupled with the underlying memory hierarchy and compute capabilities, rendering hardware-agnostic benchmarks insufficient. A primary challenge lies in characterizing the transition between memory-bound and compute-bound regimes, which varies significantly across architectures. For instance, while high-end GPUs like the NVIDIA A100 are often constrained by memory bandwidth during decoding, edge devices and CPUs face severe limitations in both cache size and interconnect throughput [112]. Consequently, profiling methodologies must isolate specific bottlenecks, such as the impact of row buffer locality in non-volatile memories or the latency of off-chip load requests, to accurately predict performance scaling [113; 114].

Cross-platform benchmarking reveals that optimizations yielding substantial gains on one architecture may degrade performance on another due to differences in cache replacement policies and bandwidth availability. Research indicates that cache management strategies like PagedAttention, while effective in eliminating fragmentation on GPUs, introduce kernel rewrite overheads that may not be portable to systems relying on OS-assisted demand paging [10]. Furthermore, the heterogeneity of modern clusters, comprising mixtures of high- and low-capacity GPUs, demands phase-aware partitioning and adaptive quantization schemes that dynamically adjust to hardware-specific constraints [72]. Without hardware-aware profiling, systems risk suboptimal resource allocation, as evidenced by studies showing that naïve scheduling on heterogeneous infrastructures can lead to significant tail latency violations and reduced goodput [115].

To address these complexities, high-fidelity simulation frameworks have emerged as critical tools for design space exploration, offering a cost-effective alternative to exhaustive real-world experimentation. Tools like [116] and [117] enable researchers to model operator performance and memory access patterns with minimal error rates, facilitating the rapid prototyping of serving configurations across thousands of hypothetical hardware setups. These simulators incorporate detailed models of memory bandwidth, cache associativity, and interconnect latency, allowing for the evaluation of techniques such as near-memory processing for recommendation systems or speculative prefetching in unified virtual memory environments [118; 119]. However, the accuracy of such simulations depends heavily on the fidelity of the underlying performance models, which must account for

dynamic workload characteristics and straggler effects inherent in distributed serving [120].

Emerging trends point toward energy-proportional profiling as a necessary extension of traditional latency and throughput metrics. As energy availability becomes a primary constraint for data center expansion, benchmarking must integrate power consumption per token and carbon emission estimates into the evaluation pipeline [121; 122]. Frameworks like [102] demonstrate that dynamic reconfiguration of cluster resources based on real-time energy-performance trade-offs can yield substantial operational savings without compromising Service Level Objectives (SLOs). Future benchmarking methodologies must therefore evolve to support multi-objective optimization, simultaneously evaluating latency, accuracy, cost, and energy efficiency across a spectrum of hardware configurations to guide the development of truly portable and sustainable LLM serving systems.

6.4 System-Level Integration and Cost-Performance Trade-off Analysis

Building on the hardware-aware profiling and energy-proportional metrics established previously, evaluating efficient LLM serving necessitates a shift from isolated component analysis to holistic system-level cost-performance assessment. While individual optimizations like PagedAttention [4] or speculative decoding [123] offer distinct benefits, their integration introduces complex non-linear interactions that dictate total cost of ownership (TCO). A rigorous evaluation framework must therefore synthesize the interplay between memory compression, algorithmic acceleration, and the hardware heterogeneity discussed earlier to determine the true economic viability of production deployments.

The primary metric for this system-level assessment is the end-to-end cost-per-token, which integrates infrastructure expenses with goodput under strict Service Level Objectives (SLOs). Disaggregated architectures, such as those separating prefill and decoding phases [33], exemplify this trade-off. While disaggregation eliminates interference and improves resource utilization, it incurs communication overheads for KV cache transfer that are sensitive to the interconnect latencies highlighted in hardware profiling. Recent studies indicate that without optimized networking, these overheads can negate latency gains, necessitating elastic memory pools like those in [101] to balance data locality against capacity scaling. Furthermore, leveraging the heterogeneity of modern clusters offers significant cost reductions; frameworks like [124] demonstrate that strategic model placement across diverse GPU tiers can reduce deployment costs by up to 77% compared to homogeneous setups, provided that latency SLOs are meticulously modeled against the specific constraints of each device tier.

Quantization and speculative decoding further complicate the cost landscape through their impact on accuracy and acceptance rates, creating a feedback loop with the memory and compute bottlenecks identified in prior sections. Aggressive low-bit quantization reduces memory footprint and bandwidth pressure but risks degrading the alignment between draft and target models, thereby lowering speculative acceptance rates and diminishing throughput gains. Conversely, adaptive strategies that modulate precision based on uncertainty [94] or dynamically adjust speculation depth [123] can maintain high goodput while minimizing computational waste. High-fidelity simulation frameworks such as [116] are critical here, enabling the exploration of vast configuration spaces—including batch sizes, parallelism strategies, and quantization schemes—to identify Pareto-optimal operating points that would be prohibitively expensive to discover via empirical trial-and-error, effectively bridging the gap between theoretical hardware models and practical economic outcomes.

Emerging trends point toward predictive, learning-augmented orchestration as the frontier of cost-

performance optimization, extending the dynamic reconfiguration concepts introduced in energy-proportional profiling. Systems must increasingly rely on runtime predictors to anticipate workload bursts and straggler behaviors, employing techniques like redundancy only when service time distributions justify the added cost [125]. Moreover, as context lengths extend into the hundreds of thousands of tokens, the cost of KV cache management becomes the dominant factor, reinforcing the need for the advanced memory strategies previously discussed. Future evaluation methodologies must therefore fully integrate energy-proportional metrics and carbon footprint analyses, moving beyond simple latency-throughput curves to assess the sustainability of massive-scale serving. Ultimately, achieving optimal cost-performance requires a co-design approach where scheduling policies, memory hierarchies, and algorithmic approximations are jointly tuned, leveraging tools like [126] adapted for LLM workloads to automate the navigation of this multi-dimensional trade-off space.

7 Conclusion

This subsection synthesizes the critical insights derived from our comprehensive survey, emphasizing that the path toward scalable Large Language Model (LLM) serving lies not in isolated optimizations, but in the synergistic co-design of Key-Value (KV) cache management, speculative decoding, and quantization. Our analysis reveals that while individual techniques such as PagedAttention [4] effectively mitigate memory fragmentation, and speculative decoding [6] successfully decouples proposal from verification, their combined application yields multiplicative rather than additive gains. For instance, integrating ultra-low-bit KV cache quantization, as demonstrated by KIVI [65] and ZipCache [28], directly expands the feasible batch size for speculative verification, thereby alleviating the memory bandwidth bottlenecks that typically constrain parallel token acceptance. However, this synergy introduces complex trade-offs; aggressive quantization can degrade the alignment between draft and target models, reducing acceptance rates and negating the latency benefits of speculation. Recent advances like QuaRot [29] and SpinQuant [30] address this by smoothing activation distributions, yet a unified theoretical framework quantifying the joint error propagation across these orthogonal dimensions remains an open challenge.

Looking forward, the field must transition from heuristic-based tuning to automated, adaptive co-design systems. The vast configuration space spanned by variable bit-widths, speculation depths, and cache eviction policies necessitates intelligent orchestration mechanisms capable of real-time adjustment under dynamic workloads. Simulation frameworks like Vidur [116] provide a foundational step for exploring these design spaces, but future research must develop online learning agents that autonomously navigate the efficiency trilemma of latency, cost, and accuracy. Furthermore, as context windows extend into the millions of tokens, disaggregated architectures [33; 35] will become paramount, requiring robust protocols for transferring compressed KV states across heterogeneous nodes without incurring prohibitive communication overheads. The emerging paradigm of “compute-optimal inference” [127] suggests that future systems may dynamically swap between larger, quantized models and smaller, speculative drafters based on task complexity, fundamentally reshaping serving infrastructures.

Finally, the implications of these efficiencies extend beyond performance metrics to sustainability and accessibility. By reducing the energy footprint per token through techniques like LUT-GEMM [78] and enabling deployment on resource-constrained edge devices [128; 129], synergistic optimization democratizes access to advanced AI capabilities. Nevertheless, ensuring robustness against adversarial attacks and preventing hallucination induced by aggressive compression [23] remains critical. As we advance, the community must prioritize the development of rigorous benchmarks that evaluate not only throughput but also the preservation of factual consistency and safety align-

ment in compressed, speculative serving environments, ensuring that the pursuit of efficiency does not compromise the reliability of generative intelligence.

References

- [1] AI and Memory Wall
- [2] Everybody Prune Now Structured Pruning of LLMs with only Forward Passes
- [3] GOBO Quantizing Attention-Based NLP Models for Low Latency and Energy Efficient Inference
- [4] Efficient Memory Management for Large Language Model Serving with PagedAttention
- [5] PyramidInfer: Pyramid KV Cache Compression for High-throughput LLM Inference
- [6] Speculative Decoding Exploiting Speculative Execution for Accelerating Seq2seq Generation
- [7] Sequoia Scalable, Robust, and Hardware-aware Speculative Decoding
- [8] REST Retrieval-Based Speculative Decoding
- [9] GEAR An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM
- [10] vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention
- [11] DéjàVu KV-cache Streaming for Fast, Fault-tolerant Generative LLM Serving
- [12] Learning Memory Access Patterns
- [13] Paging with Succinct Predictions
- [14] Paging with dynamic memory capacity
- [15] Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving
- [16] H₂O Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models
- [17] Scissorhands Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time
- [18] Attention Score is not All You Need for Token Importance Indicator in KV Cache Reduction: Value Also Matters
- [19] Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference
- [20] ThinK: Thinner Key Cache by Query-Driven Pruning
- [21] PQCache: Product Quantization-based KVCache for Long Context LLM Inference
- [22] PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling
- [23] No Token Left Behind Reliable KV Cache Compression via Importance-Aware Mixed Precision Quantization
- [24] RazorAttention: Efficient KV Cache Compression Through Retrieval Heads
- [25] Reducing Transformer Key-Value Cache Size with Cross-Layer Attention
- [26] Effectively Compress KV Heads for LLM

- [27] MiniCache: KV Cache Compression in Depth Dimension for Large Language Models
- [28] ZipCache: Accurate and Efficient KV Cache Quantization with Salient Token Identification
- [29] QuaRot Outlier-Free 4-Bit Inference in Rotated LLMs
- [30] SpinQuant: LLM quantization with learned rotations
- [31] Prefixing Attention Sinks can Mitigate Activation Outliers for Large Language Model Quantization
- [32] KV Cache is 1 Bit Per Channel: Efficient Large Language Model Inference with Coupled Quantization
- [33] DistServe Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving
- [34] Inference without Interference Disaggregate LLM Inference for Mixed Downstream Workloads
- [35] Infinite-LLM Efficient LLM Service for Long Context with DistAttention and Distributed KVCache
- [36] RAGCache Efficient Knowledge Caching for Retrieval-Augmented Generation
- [37] AttentionStore Cost-effective Attention Reuse across Multi-turn Conversations in Large Language Model Serving
- [38] Prompt Cache Modular Attention Reuse for Low-Latency Inference
- [39] ChunkAttention Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition
- [40] Redy Remote Dynamic Memory Cache
- [41] Effectively Prefetching Remote Memory with Leap
- [42] DistillSpec Improving Speculative Decoding via Knowledge Distillation
- [43] LaMini-LM A Diverse Herd of Distilled Models from Large-Scale Instructions
- [44] Medusa Simple LLM Inference Acceleration Framework with Multiple Decoding Heads
- [45] Hydra Sequentially-Dependent Draft Heads for Medusa Decoding
- [46] Kangaroo: Lossless Self-Speculative Decoding via Double Early Exiting
- [47] Inference with Reference Lossless Acceleration of Large Language Models
- [48] Ouroboros Speculative Decoding with Large Model Enhanced Drafting
- [49] Recurrent Drafter for Fast Speculative Decoding in Large Language Models
- [50] Accelerating Large Language Model Decoding with Speculative Sampling
- [51] SpecInfer Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification
- [52] Multi-Candidate Speculative Decoding
- [53] Multi-Token Joint Speculative Decoding for Accelerating Large Language Model Inference
- [54] SpecDec++: Boosting Speculative Decoding via Adaptive Candidate Lengths

- [55] Generation Meets Verification Accelerating Large Language Model Inference with Smart Parallel Auto-Correct Decoding
- [56] Optimizing Speculative Decoding for Serving Large Language Models Using Goodput
- [57] Online Speculative Decoding
- [58] Efficient Interactive LLM Serving with Proxy Model-based Sequence Length Prediction
- [59] Accelerating Retrieval-Augmented Language Model Serving with Speculation
- [60] Pythia A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning
- [61] Learning to Cache With No Regrets
- [62] QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving
- [63] Atom Low-bit Quantization for Efficient and Accurate LLM Serving
- [64] TriForce Lossless Acceleration of Long Sequence Generation with Hierarchical Speculative Decoding
- [65] KIVI A Tuning-Free Asymmetric 2bit Quantization for KV Cache
- [66] FineQuant Unlocking Efficiency with Fine-Grained Weight-Only Quantization for LLMs
- [67] OmniQuant Omnidirectionally Calibrated Quantization for Large Language Models
- [68] GPTVQ The Blessing of Dimensionality for LLM Quantization
- [69] Towards Optimizing the Costs of LLM Usage
- [70] SparQ Attention Bandwidth-Efficient LLM Inference
- [71] SqueezeLLM Dense-and-Sparse Quantization
- [72] LLM-PQ Serving LLM on Heterogeneous Clusters with Phase-Aware Partition and Adaptive Quantization
- [73] Faster and Lighter LLMs A Survey on Current Challenges and Way Forward
- [74] SnapKV LLM Knows What You are Looking for Before Generation
- [75] Mitigating Quantization Errors Due to Activation Spikes in GLU-Based LLMs
- [76] QAQ Quality Adaptive Quantization for LLM KV Cache
- [77] MARLIN: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models
- [78] LUT-GEMM Quantized Matrix Multiplication based on LUTs for Efficient Inference in Large-Scale Generative Language Models
- [79] ZeroQuant Efficient and Affordable Post-Training Quantization for Large-Scale Transformers
- [80] Efficient 8-Bit Quantization of Transformer Neural Machine Language Translation Model
- [81] FlashDecoding++ Faster Large Language Model Inference on GPUs
- [82] Efficient Execution of Quantized Deep Learning Models A Compiler Approach
- [83] QLoRA Efficient Finetuning of Quantized LLMs

- [84] QA-LoRA Quantization-Aware Low-Rank Adaptation of Large Language Models
- [85] L4Q Parameter Efficient Quantization-Aware Training on Large Language Models via LoRA-wise LSQ
- [86] EfficientQAT: Efficient Quantization-Aware Training for Large Language Models
- [87] BitDistiller Unleashing the Potential of Sub-4-Bit LLMs via Self-Distillation
- [88] OneBit Towards Extremely Low-bit Large Language Models
- [89] Slim-LLM: Saliency-Driven Mixed-Precision Quantization for Large Language Models
- [90] Any-Precision LLM Low-Cost Deployment of Multiple, Different-Sized LLMs
- [91] The case for 4-bit precision k-bit Inference Scaling Laws
- [92] What Makes Quantization for Large Language Models Hard An Empirical Study from the Lens of Perturbation
- [93] InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management
- [94] RT-LM Uncertainty-Aware Resource Management for Real-Time Inference of Language Models
- [95] Reconciling High Accuracy, Cost-Efficiency, and Low Latency of Inference Serving Systems
- [96] COCOA Cold Start Aware Capacity Planning for Function-as-a-Service Platforms
- [97] LaSS Running Latency Sensitive Serverless Computations at the Edge
- [98] SLAM SLO-Aware Memory Optimization for Serverless Applications
- [99] EdgeShard: Efficient LLM Inference via Collaborative Edge Computing
- [100] Serving and Optimizing Machine Learning Workflows on Heterogeneous Infrastructures
- [101] MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool
- [102] DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency
- [103] ServerlessLLM Locality-Enhanced Serverless Inference for Large Language Models
- [104] Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve
- [105] Predictive Pipelined Decoding A Compute-Latency Trade-off for Exact LLM Decoding
- [106] Andes Defining and Enhancing Quality-of-Experience in LLM-Based Text Streaming Services
- [107] Evaluating Quantized Large Language Models
- [108] LongLLMLingua Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression
- [109] tinyBenchmarks evaluating LLMs with fewer examples
- [110] Rethinking LLM Memorization through the Lens of Adversarial Compression
- [111] Decoding Compressed Trust Scrutinizing the Trustworthiness of Efficient LLMs Under Compression
- [112] MELTing point Mobile Evaluation of Language Transformers

- [113] Evaluating Row Buffer Locality in Future Non-Volatile Main Memories
- [114] Hermes Accelerating Long-Latency Load Requests via Perceptron-Based Off-Chip Load Prediction
- [115] Llumnix: Dynamic Scheduling for Large Language Model Serving
- [116] Vidur: A Large-Scale Simulation Framework For LLM Inference
- [117] A Hardware Evaluation Framework for Large Language Model Inference
- [118] RecNMP Accelerating Personalized Recommendation with Near-Memory Processing
- [119] Deep Learning based Data Prefetching in CPU-GPU Unified Virtual Memory
- [120] START Straggler Prediction and Mitigation for Cloud Computing Environments using Encoder LSTM Networks
- [121] Towards Greener LLMs Bringing Energy-Efficiency to the Forefront of LLM Inference
- [122] From Words to Watts Benchmarking the Energy Costs of Large Language Model Inference
- [123] MagicDec: Breaking the Latency-Throughput Tradeoff for Long Context Generation with Speculative Decoding
- [124] Mélange Cost Efficient Large Language Model Serving by Exploiting GPU Heterogeneity
- [125] Straggler Mitigation at Scale
- [126] Llama A Heterogeneous & Serverless Framework for Auto-Tuning Video Analytics Pipelines
- [127] An Empirical Analysis of Compute-Optimal Inference for Problem-Solving with Language Models
- [128] PowerInfer-2: Fast Large Language Model Inference on a Smartphone
- [129] EdgeMoE Fast On-Device Inference of MoE-based Large Language Models